

**Módulo: Programación Multimedia y dispositivos móviles.**  
**Unidad de Trabajo N°2: Desarrollo de aplicaciones Android.**

**Actividad de Clase 1: Introducción a JetPack Compose.**

**1. Objetivo general.**

Crear nuestra primera App con JetPack Compose.

**2. Metodología.**

Cada alumno creará una app utilizando los controles básicos Text, TextField y Button en Compose.

**3. Descripción.**

Realiza lo siguiente:

**Primera Parte: Introducción a JetPack Compose (Column(), Row(), LazyColumn())**

1. Vamos a crear tu primera App con Compose
2. Una vez creado eliminaremos el código que crea el asistente para añadir nuestro primer código. Crearemos una función **@Composable** y también una función **@Preview** que nos permitirá ver los cambios en nuestra app sin necesidad de ejecutar el emulador cada vez que realizamos algún cambio en el código. También vamos a modificar las opciones de Preview para ver como se ve con diferentes opciones. Para recordar como se crea el proyecto y ver como introducimos nuestro primer código en JetPack Compose podemos observar el siguiente video:

<https://drive.google.com/file/d/1VPE1qnQbvffUTuqluv9FRzIZVIVGiMlj/view?usp=sharing>

3. Ahora vamos a ver como se añade más texto y como lo podemos ordenar para que aparezca de forma vertical (cada uno debajo del anterior) utilizando la función Composable **Column()**

[https://drive.google.com/file/d/1FyTNjy2y3Ssr6V2rHf7a7fA2tkS3bs3\\_/view?usp=sharing](https://drive.google.com/file/d/1FyTNjy2y3Ssr6V2rHf7a7fA2tkS3bs3_/view?usp=sharing)

4. En el siguiente video puedes observar como podemos mezclar filas **Row()** y columnas para que lo que mostramos tenga un aspecto diferente, utilizando algunos modificadores para la imagen. Puedes ver como realizarlo consultando el siguiente video:

<https://drive.google.com/file/d/1PCFJYZ169DcjcfxNKX5ltsxxsAWB42uk/view?usp=sharing>

5. Una vez visto las formas básicas para mostrar diferentes componentes, podemos observar como mostrar un listado vertical al estilo RecyclerView utilizando **LazyColumn()**. Pero de una forma más sencilla.

<https://drive.google.com/file/d/1cXB6lzUWwrypJr0JWUTSI5bICBg-m3Rq/view?usp=sharing>

6. Con los componentes visto crea una pequeña app que los utilice.

## Segunda Parte: Entrada de textos (TextField, OutlinedTextField y Button)

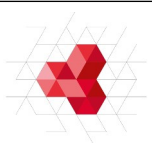
Vamos a crear una pequeña App para que el usuario introduzca su nombre de usuario y su contraseña. Y cuando pulse el botón mostrará en un Text un mensaje de error si no se ha cargado el nombre o la contraseña tiene menos de 8 caracteres. Con TextField y OutlinedTextField (es el que utilizaremos), vamos a poder entrar datos y con el botón confirmar. El aspecto de la App puede ser parecido al siguiente:



7. Vamos a crear tu App con Compose
8. Una vez creado eliminaremos el código que crea el asistente para añadir nuestro primer código colocando el nombre de la función Composable que vamos a crear.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            Login()  
        }  
    }  
}
```

9. Crearemos una función **@Composable** que en este ejemplo he llamado **Login()**.
10. Aquí puedes ver el código de la función (Composable) Login.



## Actividad de Desarrollo PGL



```
@Composable
fun Login() {
    Column (
        //Modificadores para que ocupe todo el espacio y es centrando verticalmente
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center
    ) { this: ColumnScope
        //Definimos la variables de las que queremos guardar su estado
        var usuario by remember { mutableStateOf( value: "" ) }
        var clave by remember { mutableStateOf( value: "" ) }
        var resultado by remember { mutableStateOf( value: "Sin respuesta") }
        OutlinedTextField(
            //Valor
            value = usuario
            //Nuevo valor
            , onValueChange = { usuario = it },
            //Etiqueta
            label = { Text( text: "Nombre de usuario") },
            //Modificadores ancho, el espaciado y una sólo línea.
            modifier = Modifier
                .fillMaxWidth()
                .padding(10.dp),
            singleLine = true
        )
        OutlinedTextField(
            //Valor
            value = clave,
            //Nuevo valor
            onValueChange = {clave = it },
            //Modificadores ancho, el espaciado, una sólo línea y que no se muestre.
            modifier = Modifier
                .fillMaxWidth()
                .padding(10.dp),
            singleLine = true,
            visualTransformation = PasswordVisualTransformation()
        )
    }
}
```

```
Button(
    onClick = {
        var cadena = ""
        if (clave.length < 8)
            cadena += "La clave debe tener al menos 8 caracteres"
        if (usuario.length == 0)
            cadena += "No se puede dejar el usuario vacío"
        resultado = cadena
    },
    modifier = Modifier.padding(10.dp)
) { this: RowScope
    Text(text = "Confirmar")
}
Text(
    text = resultado,
    modifier = Modifier.padding(10.dp)
)
}
```

11. Breve explicación del código: Para mostrar el cuadro de entrada de datos llamamos a la función composable **'OutlinedTextField'** y le pasamos una serie de datos:

Los dos parámetros fundamentales de la función `OutlinedTextField` son el `value` que le pasamos una variable previamente creada llamado a la función `remember`:

**`var usuario by remember { mutableStateOf("") }`**

La variable `'usuario'` almacena los datos ingresados por teclado por el usuario, los mismos se actualizan según la función `lambda` enviada al parámetro `onValueChange`.

Los dos parámetros: **'value'** y **'onValueChange'** debemos especificarlos para que funcione nuestro editor de línea, los demás parámetros como `'label'`, `'modifier'`, `'singleLine'` etc. son opcionales.

```
OutlinedTextField(  
    //Valor  
    value = usuario  
    //Nuevo valor  
    , onValueChange = { usuario = it },  
    //Etiqueta  
    label = { Text( text: "Nombre de usuario") },  
    //Modificadores ancho, el espaciado y una sola línea.  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(10.dp),  
    singleLine = true  
)
```

De forma similar trabajamos con la entrada de la clave, primero definimos la variable a partir de la llamada a la función `remember`.

La llamada a la función composable **'Button'** requiere como mínimo especificar el parámetro **onClick** con la función `lambda` que implementará la lógica de validación del ingreso de datos:

```
Button(  
    onClick = {  
        var cadena = ""  
        if (clave.length < 8)  
            cadena += "La clave debe tener al menos 8 caracteres\n"  
        if (usuario.length == 0)  
            cadena += "No se puede dejar el usuario vacío"  
        resultado = cadena  
    },  
    modifier = Modifier.padding(10.dp)  
) { this: RowScope  
    Text(text = "Confirmar")  
}
```

Mediante la variable 'resultado', previamente definida:

**var resultado by remember {mutableStateOf("Sin resultado")}**

Al asignar en el algoritmo codificado en el 'onClick':

**resultado = cadena**

Se actualizar en forma automática el texto que muestra en la llamada a la función Text:

```
Text(  
    text = resultado,  
    modifier = Modifier.padding(10.dp)  
)
```

12. También llamaremos a dicha función en la función **@Preview** que nos permitirá ver los cambios en nuestra app sin necesidad de ejecutar el emulador cada vez que realizamos algún cambio en el código.

```
@Preview(showSystemUi = true)  
@Composable  
fun GreetingPreview() {  
    Login()  
}
```

13. En último lugar, podemos ver un video del funcionamiento de la App.

[https://drive.google.com/file/d/15f4wJK1qJlOLd9\\_-ztOr8I\\_3G\\_0bT\\_1h/view?usp=sharing](https://drive.google.com/file/d/15f4wJK1qJlOLd9_-ztOr8I_3G_0bT_1h/view?usp=sharing)